

Documento Técnico: Prueba de Concepto (PoC) y Estrategia de Evolución hacia un MVP para Integración de Pasarela de Pagos

Introducción y Objetivos del Proyecto

En el desarrollo de sistemas de software modernos, la automatización del procesamiento de pagos es un componente crítico que impacta directamente en la consistencia de los datos y en la experiencia del usuario. El objetivo de este proyecto fue diseñar e implementar una **Prueba de Concepto (PoC)** de un flujo de pagos asíncrono y automatizado.

Este documento detalla la arquitectura actual del prototipo local (Parte 1) y define la hoja de ruta técnica para su evolución hacia un **Producto Mínimo Viable (MVP)** escalable, seguro y desplegado en la nube de Microsoft Azure (Parte 2).

PARTE 1: Arquitectura y Funcionamiento de la Prueba de Concepto (PoC)

La PoC se diseñó como un entorno local cerrado enfocado en validar la viabilidad técnica del procesamiento de eventos asíncronos distribuidos.

1.1. Componentes del Entorno Local

El ecosistema actual del prototipo está integrado por las siguientes herramientas:

- **Backend (Flask):** Servidor local encargado de interactuar con el SDK de Mercado Pago para generar preferencias de compra y exponer el puerto receptor de alertas.
- **Base de Datos (PostgreSQL):** Almacenamiento relacional local donde reside la tabla `pedidos` con estados dinámicos (`PENDIENTE`, `PAGADO`).
- **Túnel de Red (ngrok):** Herramienta de reenvío de puertos que expone de forma pública e interna el puerto local `8080` bajo un protocolo seguro (`https`), permitiendo la comunicación desde servidores externos hacia la laptop de desarrollo.

1.2. Mecanismo del Webhook y el Payment Listener

El núcleo de la automatización radica en romper el esquema tradicional de consulta constante (*API Polling*) para implementar una arquitectura orientada a eventos (*Event-Driven Architecture*).

- **El Webhook (El Emisor):** Es el canal de comunicación que utiliza la pasarela de pagos. Cuando ocurre una transacción exitosa, Mercado Pago emite proactivamente una petición HTTP con el método `POST` conteniendo un JSON con los metadatos del evento (`action: payment.updated, id: del_pago`).
- **El Payment Listener (El Receptor):** Es el componente lógico programado dentro del backend de Flask bajo la ruta `@app.route('/api/v1/webhook', methods=['POST'])`. Su única función es mantenerse a la escucha en el puerto expuesto. Cuando recibe la petición `POST` del Webhook, el Listener se activa de forma asíncrona, consume el JSON, valida la autenticidad del pago con el SDK, realiza el `UPDATE` en la base de datos PostgreSQL pasando el estado del pedido a `PAGADO` y ejecuta las acciones secundarias.

1.3. El Entorno de Pruebas de Mercado Pago (Sandbox)

Para simular el flujo financiero sin transaccionar dinero real, se utilizó el entorno **Sandbox**. Este ecosistema emula el comportamiento de producción pero utiliza usuarios compradores ficticios, saldos virtuales y métodos de pago simulados (tarjetas de prueba o Yapes simulados).

- **Forzado de URL en Sandbox (`notification_url`):** En entornos de desarrollo Checkout Pro suele ignorar las configuraciones globales de webhooks de la interfaz web. Por ello, se implementó de forma dinámica el parámetro "`notification_url`" directamente en el diccionario de creación de la preferencia de pago en Python. Esto fuerza a la pasarela a redirigir el tráfico del Webhook al dominio temporal provisto por ngrok de manera infalible.

1.4. Flujo de Notificación por Correo Electrónico Temporal

Una vez que el *Payment Listener* confirma el pago e impacta la base de datos, se gatilla la fase de mensajería utilizando la librería nativa de Python `smtplib`. Esta librería se conecta de forma segura mediante protocolo TLS al servidor SMTP de Gmail (`smtp.gmail.com:587`). Para saltar las barreras de autenticación multifactor de Google, se implementó una **Contraseña de Aplicación de 16 caracteres**, permitiendo al script despachar un comprobante de pago con diseño HTML estructurado y el monto exacto cobrado (`transaction_amount`).

PARTE 2: Evolución hacia el MVP y Despliegue en la Nube (Azure)

La transición de una PoC local a un MVP comercial implica migrar la infraestructura hacia un entorno de producción que garantice alta disponibilidad, tolerancia a fallos y seguridad empresarial.

2.1. Importancia del Despliegue en Producción

Tener el proyecto desplegado en la nube es indispensable por las siguientes razones:

- **Disponibilidad 24/7:** El sistema deja de depender de que la laptop del desarrollador esté encendida o que el túnel de ngrok expire.
- **Punto de Acceso Público Real:** Desaparece ngrok. La aplicación obtiene un dominio propio protegido con certificados SSL válidos, lo cual es un requisito obligatorio para operar en el modo productivo real de cualquier pasarela de pagos.
- **Persistencia y Escalabilidad:** Los datos y la lógica se almacenan en centros de datos distribuidos que soportan aumentos drásticos en la concurrencia de usuarios.

2.2. Migración de Infraestructura (Cómputo en Azure)

En el MVP, el código de Python se hospedará en la nube utilizando uno de los siguientes modelos de servicio de Azure:

- **Azure App Service (Recomendado - PaaS):** Permite subir la aplicación Flask completa a un entorno administrado por Microsoft. Azure otorga una URL fija (ej. `https://mi-tienda.azurewebsites.net`). El *Payment Listener* seguirá residiendo en la misma ruta del código, y la URL del webhook en Mercado Pago se actualizará apuntando de forma directa al nuevo dominio de Azure.

- **Azure Functions (Alternativa Serverless):** Consiste en aislar la función del *Payment Listener* en un microservicio independiente que se ejecuta únicamente cuando Mercado Pago envía un HTTP POST. Esto abarata costos ya que solo se paga por milisegundo de ejecución cuando hay compras reales.

2.3. Transición del Motor de Base de Datos a Azure SQL Database

En producción, PostgreSQL local se reemplaza por **Azure SQL Database**, un servicio PaaS que ejecuta el motor relacional de **Microsoft SQL Server**.

- **Cambio Tecnológico en Python:** Se sustituye la librería `psycopg2` por `pyodbc`, utilizando una cadena de conexión segura (`Connection String`) cifrada hacia el servidor de Azure por el puerto 1433.
- **Control de Accesos (Firewall):** Azure SQL Database viene bloqueado por defecto. Se requiere configurar reglas de Firewall en el portal de Azure para permitir que el rango de IPs de nuestro *App Service* tenga permisos de lectura/escritura, así como añadir la IP pública del administrador para la gestión remota desde herramientas como **DBeaver**.

2.4. Rediseño del Flujo de Notificaciones (Lógica de Negocio Doble)

El script de correo básico de la PoC evoluciona hacia una solución transaccional de producción:

- **Servicio de Correo en la Nube (SendGrid o Azure Communication Services):** Se abandona el SMTP clásico de Gmail para evitar bloqueos por volumen y filtros de SPAM. Se consumen APIs dedicadas que aseguran la entrega masiva e inmediata de correos.
- **Lógica de Doble Notificación Simultánea:** El Webhook Listener en Azure procesará el evento y disparará dos flujos de correo paralelos con objetivos de negocio diferenciados:
 1. **Correo para el Comprador (Cliente):** Envío automático del comprobante con el ID de transacción, el monto en soles (S/.) cobrado y un mensaje de éxito.
 2. **Correo para el Vendedor (Administrador/Almacén):** Alerta interna instantánea que notifica al equipo de operaciones que un pedido específico ha sido pagado y que debe iniciarse el proceso de preparación y despacho de la mercadería.